UNIVERSIDAD CATÓLICA DE CUENCA

FACULTAD DE INFORMÁTICA, CIENCIAS DE LA COMPUTACIÓN E INNOVACIÓN TECNOLÓGICA

CARRERA DE ROBÓTICA E INTELIGENCIA ARTIFICIAL

ASIGNATURA: MICROCONTROLADORES

TEMA: SISTEMA DE CONTROL DE TIEMPO REAL PARA CÉLULA ROBOTICA

NOMBRE DEL ESTUDIANTE:

Mateo Roca

DOCENTE: ING. PABLO BUESTÁN ANDRADE, PhD

FECHA: 1/05/2026

INTRODUCCIÓN

La sincronización entre eventos externos y procesos temporizados es el núcleo de la robótica industrial. Los sistemas de tiempo real deben ser capaces de atender interrupciones de sensores mientras ejecutan rutinas de movimiento precisas. Esta práctica implementa una lógica de control de procesos mediante el uso de temporizadores de hardware en el ESP32, eliminando la latencia producida por el software convencional y garantizando una ejecución determinista.

OBJETIVO GENERAL

Desarrollar una máquina de estados compleja para una célula de manufactura automatizada, utilizando interrupciones externas y timers de hardware para la gestión de procesos secuenciales asíncronos.

DESARROLLO

Configuración de interrupciones externas

Para cumplir con el requisito de respuesta inmediata ante eventos asíncronos, se configuraron tres pulsantes utilizando la función attachInterrupt(). Estos botones simulan sensores industriales en la planta:

- **Sensor de Detección (Pin 13):** Activa el paso de la pieza a la zona de espera.
- **Sensor de Clasificación (Pin 12):** Inicia el proceso de trabajo tras la decisión de la IA.
- **Botón de Reinicio (Pin 14):** Permite al operario resetear el sistema una vez entregado el producto. Se utilizó el modo FALLING y la resistencia interna PULLUP del ESP32 para garantizar lecturas limpias sin ruido eléctrico.

Gestión de tiempo con timer de harware

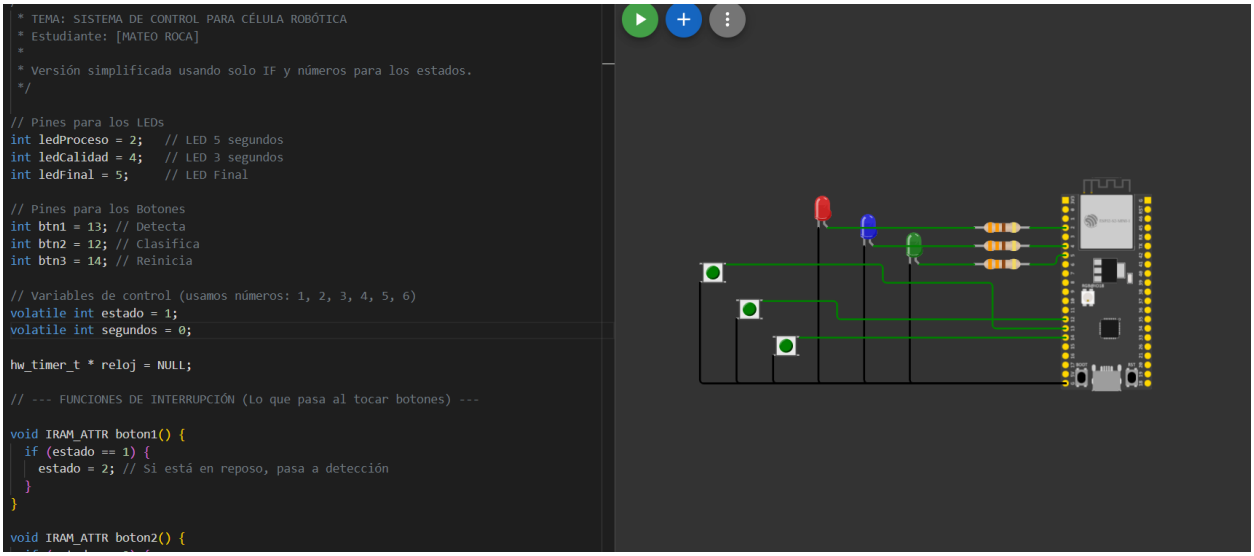
En lugar de utilizar funciones que bloquean el procesador como delay(), se implementó un objeto de tipo hw_timer_t. El timer se configuró con un divisor de frecuencia de 80 para obtener una base de tiempo de microsegundos, generando una interrupción exacta cada 1,000,000 de pulsos (1 segundo). Esta técnica permite que el microcontrolador gestione los tiempos de ensamblaje (5s) e inspección (3s) con precisión milimétrica, permitiendo que el sistema atienda otras tareas en paralelo.

Lógica de maquina de estados

El control del sistema se basa en una variable numérica que recorre 6 etapas lógicas:

1. **Estado 1 (Reposo):** Todo el sistema apagado.
2. **Estado 2 (Detección):** Confirmación de pieza en banda.
3. **Estado 3 (Clasificación):** Decisión del flujo de trabajo.
4. **Estado 4 (Ensamblaje):** Activación del actuador robótico (LED Proceso) por 5 segundos.
5. **Estado 5 (Inspección):** Validación de calidad por visión artificial (LED Inspección) por 3 segundos.
6. **Estado 6 (Listo):** Indicación de producto terminado y espera de liberación.

DISEÑO ESQUEMATICO



CODIGO

```
/*
 * TEMA: SISTEMA DE CONTROL PARA CÉLULA ROBÓTICA
 * Estudiante: [MATEO ROCA]
 *
 * Versión simplificada usando solo IF y números para los estados.
 */

// Pines para los LEDs
int ledProceso = 2;   // LED 5 segundos
int ledCalidad = 4;   // LED 3 segundos
int ledFinal = 5;     // LED Final

// Pines para los Botones
int btn1 = 13; // Detecta
int btn2 = 12; // Clasifica
int btn3 = 14; // Reinicia

// Variables de control (usamos números: 1, 2, 3, 4, 5, 6)
volatile int estado = 1;
volatile int segundos = 0;

hw_timer_t * reloj = NULL;

// --- FUNCIONES DE INTERRUPCIÓN (Lo que pasa al tocar botones) ---
```

```
void IRAM_ATTR boton1() {
  if (estado == 1) {
    estado = 2; // Si está en reposo, pasa a detección
  }
}

void IRAM_ATTR boton2() {
  if (estado == 2) {
    estado = 3; // Si detectó, pasa a clasificar
  }
}

void IRAM_ATTR boton3() {
  if (estado == 6) {
    estado = 1; // Si terminó, reinicia a reposo
  }
}

// --- FUNCIÓN DEL TIMER (Cuenta 1 segundo) ---
void IRAM_ATTR cuentaTiempo() {
  segundos++;
}

void setup() {
  Serial.begin(115200);

  pinMode(ledProceso, OUTPUT);
  pinMode(ledCalidad, OUTPUT);
  pinMode(ledFinal, OUTPUT);

  pinMode(btn1, INPUT_PULLUP);
  pinMode(btn2, INPUT_PULLUP);
  pinMode(btn3, INPUT_PULLUP);

  // Configurar interrupciones de los botones
  attachInterrupt(digitalPinToInterrupt(btn1), boton1, FALLING);
  attachInterrupt(digitalPinToInterrupt(btn2), boton2, FALLING);
  attachInterrupt(digitalPinToInterrupt(btn3), boton3, FALLING);

  // Configurar el Timer para que cuente cada 1 segundo
  reloj = timerBegin(1000000);
  timerAttachInterrupt(reloj, &cuentaTiempo);
  timerAlarm(reloj, 1000000, true, 0);
}
```

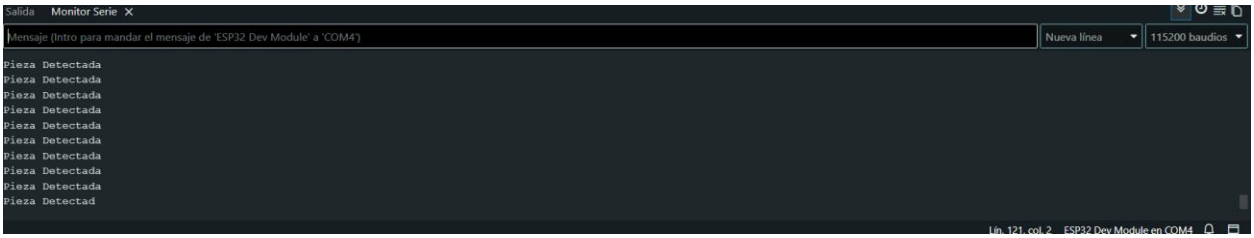
```
void loop() {

  // ESTADO 1: REPOSO
  if (estado == 1) {
    digitalWrite(ledProceso, LOW);
    digitalWrite(ledCalidad, LOW);
    digitalWrite(ledFinal, LOW);
    segundos = 0;
  }

  // ESTADO 2: DETECCIÓN
  if (estado == 2) {
    Serial.println("Pieza Detectada");
  }

  // ESTADO 3: CLASIFICACIÓN
  if (estado == 3) {
    Serial.println("Clasificando...");
    segundos = 0; // Ponemos el reloj en cero antes de empezar
    estado = 4;   // Pasamos a procesar automáticamente
  }

  // ESTADO 4: ENSAMBLAJE (Dura 5 segundos)
  if (estado == 4) {
    digitalWrite(ledProceso, HIGH);
    Serial.print("Ensamblando pieza: "); Serial.println(segundos);

    if (segundos >= 5) {
      digitalWrite(ledProceso, LOW);
      segundos = 0; // Limpiamos el tiempo para el siguiente paso
      estado = 5;   // Pasamos a inspección
    }
  }

  // ESTADO 5: CALIDAD (Dura 3 segundos)
  if (estado == 5) {
    digitalWrite(ledCalidad, HIGH);
    Serial.print("Inspeccionando calidad: "); Serial.println(segundos);

    if (segundos >= 3) {
      digitalWrite(ledCalidad, LOW);
      estado = 6; // Pasamos a listo
    }
  }
```

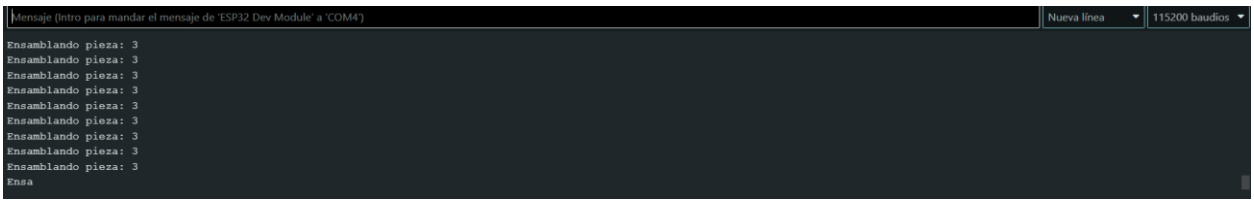
```
  // ESTADO 6: LISTO
  if (estado == 6) {
    digitalWrite(ledFinal, HIGH);
    Serial.println("Sistema Listo. Presione Reiniciar.");
  }
}
```

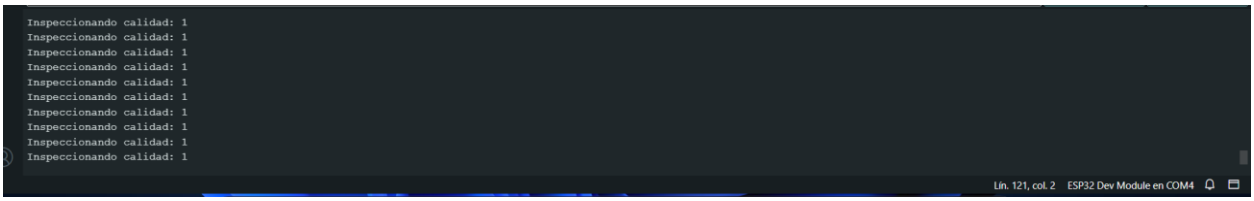
CONCLUSIONES

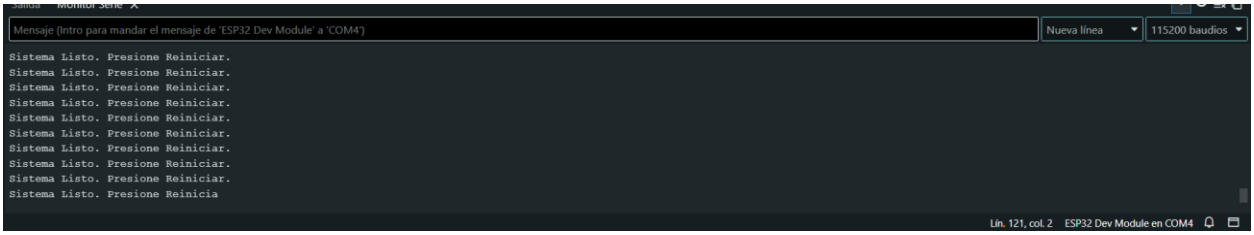
- Se logró implementar una célula robótica autónoma donde el control del tiempo no depende del software, sino del hardware interno del ESP32, asegurando que los procesos de manufactura duren exactamente lo programado sin retrasos por otros cálculos del programa.
- El uso de interrupciones externas demostró ser fundamental para la seguridad y eficiencia industrial, ya que el sistema es capaz de detectar la llegada de material al instante, sin necesidad de estar preguntando constantemente el estado del botón en el código principal.
- La estructura de máquina de estados permitió un flujo de trabajo organizado y seguro, impidiendo que el robot intente inspeccionar una pieza que aún no ha sido ensamblada, garantizando así la integridad del proceso de fabricación.

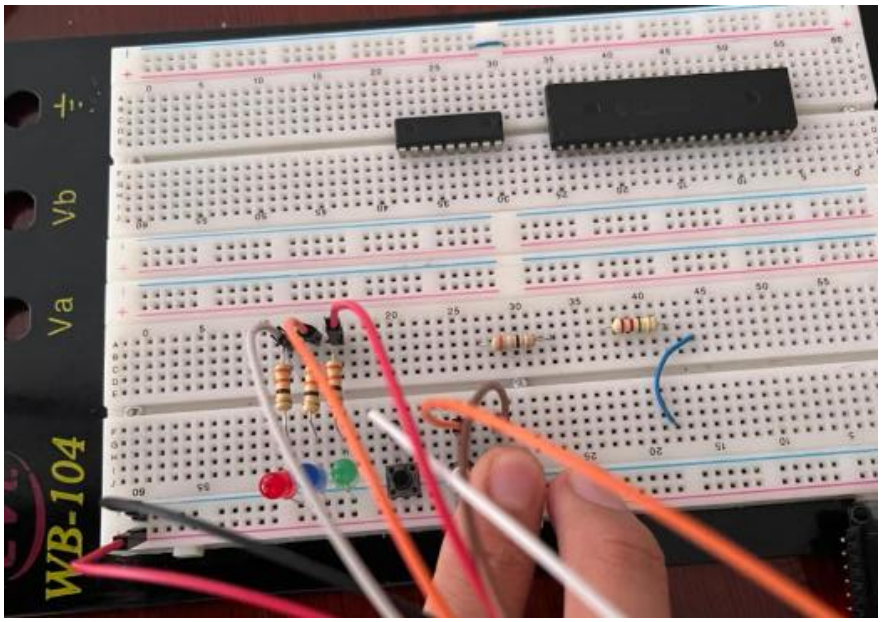
FOTOGRAFIAS

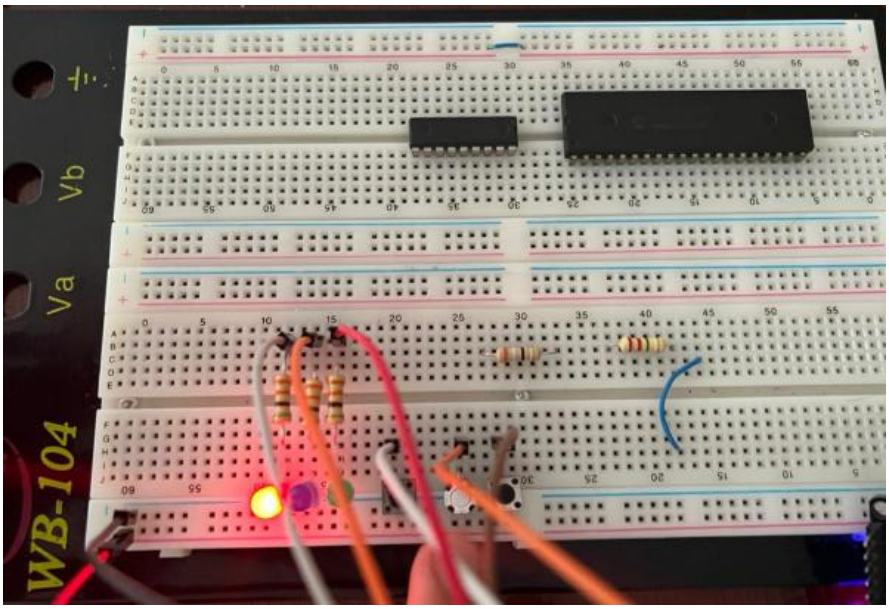


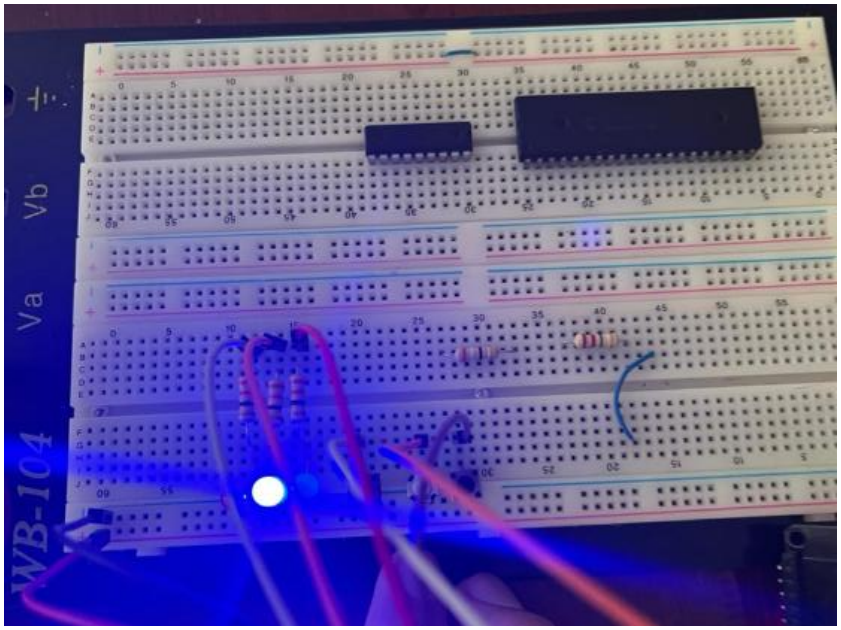


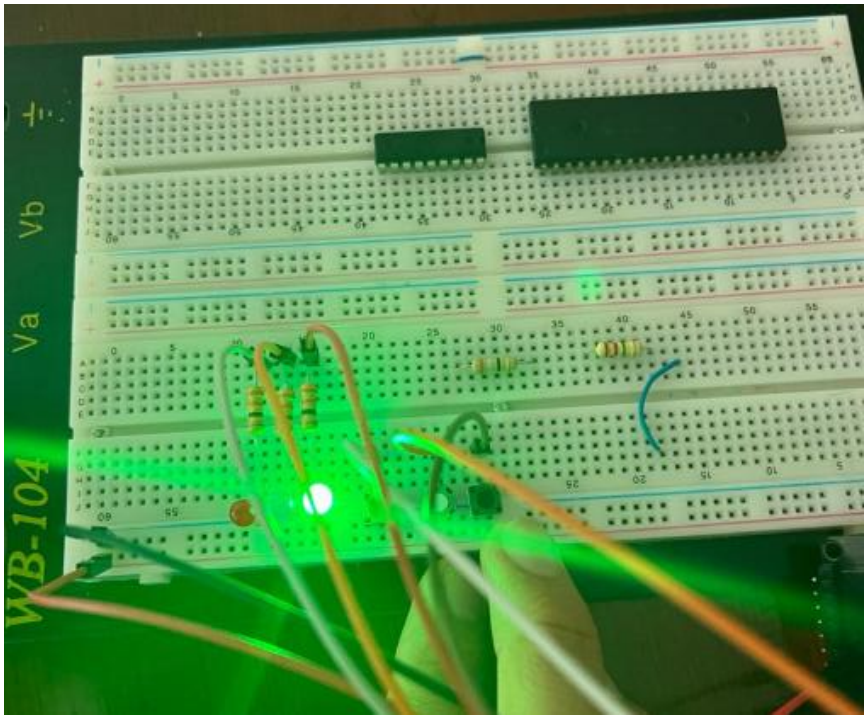



Mensaje (Intro para mandar el mensaje de 'ESP32 Dev Module' a 'COM4')
Nueva línea
115200 baudios
Sistema Listo. Presione Reiniciar.
Sistema Listo. Presione Reiniciar.
Sistema Listo. Presione Reiniciar.
Sistema Listo. Presione Reiniciar.
Sistema Listo. Presione Reiniciar.
Sistema Listo. Presione Reiniciar.
Sistema Listo. Presione Reiniciar.
Sistema Listo. Presione Reiniciar.
Sistema Listo. Presione Reiniciar.
Sistema Listo. Presione Reinicia
Lín. 121, col. 2
ESP32 Dev Module en COM4


Vb
Va
WB-104


Vb
Va
WB-104


Vb
Va
WB-104


Vb
Va
WB-104